

Universidad del Valle de Guatemala
Programación Orientada a Objetos
Arnoldo Caleb Carías Donis
Carné 261937



Análisis-Laboratorio #2

Arnoldo Caleb Carias Donis
Programación Orientada a Objetos

Requisitos funcionales

1. Solicitar al iniciar el programa el nombre, código de identificación y coordinador del festival.
2. Permitir crear un nuevo festival, eliminando los escenarios y artistas registrados anteriormente.
3. Almacenar un máximo de cinco escenarios mediante un arreglo básico de objetos.
4. Guardar el código, nombre, ubicación, capacidad máxima y estado de cada escenario.
5. Validar que la capacidad máxima de un escenario sea mayor que cero.
6. Configurar escenarios únicamente en posiciones válidas y disponibles del arreglo.
7. Verificar si una posición contiene un escenario o el valor null antes de utilizarla.
8. Consultar todos los escenarios configurados y mostrar las posiciones que ocupan.
9. Consultar un escenario específico mediante su posición.
10. Modificar la capacidad máxima y el estado de un escenario existente.
11. Retirar un escenario y dejar su posición nuevamente con el valor null.
12. Almacenar los artistas mediante un `ArrayList<Artista>`.
13. Guardar el código, nombre artístico, género musical, duración y cantidad estimada de asistentes de cada artista.
14. Validar que la duración sea mayor que cero y que la cantidad estimada de asistentes no sea negativa.
15. Registrar artistas sin permitir códigos repetidos.
16. Consultar todos los artistas y manejar correctamente cuando el `ArrayList` esté vacío.
17. Buscar artistas por su código recorriendo el `ArrayList`.
18. Modificar la información de un artista utilizando las mismas validaciones de su creación.
19. Cancelar la participación de un artista eliminándolo del `ArrayList`.
20. Mostrar la cantidad de escenarios configurados, espacios disponibles y el escenario de mayor capacidad.
21. Mostrar la cantidad de artistas, el artista de mayor duración, el de mayor asistencia estimada y el promedio de duración.
22. Evitar cálculos que necesiten elementos cuando el `ArrayList` esté vacío.
23. Manejar entradas numéricas incorrectas mediante `InputMismatchException`.
24. Manejar valores inválidos mediante `IllegalArgumentException`.
25. Utilizar bloques try-catch y al menos un bloque finally.
26. Permitir que el programa continúe funcionando después de manejar una entrada incorrecta o excepción.
27. Proteger los atributos mediante modificadores de visibilidad adecuados.
28. Mostrar un menú con las trece opciones indicadas y terminar únicamente cuando el usuario seleccione "Salir".

Clase Escenario

La clase Escenario representa cada escenario configurado dentro del festival. Su propósito es almacenar su código, nombre, ubicación, capacidad máxima y estado, además de validar y permitir la modificación de los datos indicados por el ejercicio.

Tabla de atributos

Propiedad	Tipo	Visibilidad	Propósito
código	String	private	Almacenar el código que identifica al escenario.
nombre	String	private	Almacenar el nombre asignado al escenario.
ubicación	String	private	Almacenar la ubicación física del escenario dentro del festival.
capacidadMaxima	int	private	Almacenar la cantidad máxima de asistentes permitida. Su valor debe ser mayor que 0.
estado	String	private	Almacenar el estado actual del escenario, por ejemplo: disponible, ocupado o en mantenimiento.

Clase Escenario

Tabla de Métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
Escenario()	-	String codigo, String nombre, String ubicacion, int capacidadMaxima, String estado	public	Construir e inicializar un escenario con la información proporcionada. Validará que los textos no estén vacíos y que la capacidad sea mayor que 0. Si los datos son inválidos, generará una <code>IllegalArgumentException</code> .
getCodigo()	String	Ninguno	public	Obtener el código del escenario.
getNombre()	String	Ninguno	public	Obtener el nombre del escenario.
getUbicacion()	String	Ninguno	public	Obtener la ubicación del escenario.
getCapacidadMaxima()	int	Ninguno	public	Obtener la capacidad máxima de asistentes del escenario.
getEstado()	String	Ninguno	public	Obtener el estado actual del escenario.
setCapacidadMaxima()	void	int nuevaCapacidad	public	Modificar la capacidad máxima. Si el nuevo valor es menor o igual que 0, generará una <code>IllegalArgumentException</code> .
setEstado()	void	String nuevoEstado	public	Modificar el estado del escenario. Si el nuevo estado está vacío, generará una <code>IllegalArgumentException</code> .
toString()	String	Ninguno	public	Devolver en forma de texto toda la información del escenario para poder mostrarla al usuario.

Clase Artista

La clase Artista representa a cada artista participante del festival. Su propósito es almacenar la información del artista y su presentación, además de validar que la duración y la cantidad estimada de asistentes sean correctas.

Tabla de atributos

Propiedad	Tipo	Visibilidad	Propósito
codigo	String	private	Almacenar el código que identifica al artista dentro del festival. Este código no podrá repetirse.
nombreArtistico	String	private	Almacenar el nombre artístico del participante.
generoMusical	String	private	Almacenar el género musical interpretado por el artista.
duracionPresentacion	int	private	Almacenar la duración de la presentación en minutos. Debe ser mayor que 0.
cantidadEstimadaAsistentes	int	private	Almacenar la cantidad estimada de asistentes. No puede ser negativa.

Clase Artista

Tabla de métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
Artista()	-	String codigo, String nombreArtístico, String generoMusical, int duracionPresentacion, int cantidadEstimadaAsistentes	public	Construir e inicializar un artista con la información proporcionada. Validará los textos, la duración y la cantidad estimada de asistentes. Los valores inválidos generarán una <code>IllegalArgumentException</code> .
getCodigo()	String	Ninguno	public	Obtener el código del artista.
getNombreArtístico()	String	Ninguno	public	Obtener el nombre artístico del participante.
getGeneroMusical()	String	Ninguno	public	Obtener el género musical del artista.
getDuracionPresentacion()	int	Ninguno	public	Obtener la duración de la presentación en minutos.
getCantidadEstimadaAsistentes()	int	Ninguno	public	Obtener la cantidad estimada de asistentes.
actualizarInformacion()	void	String nuevoCodigo, String nuevoNombreArtístico, String nuevoGeneroMusical, int nuevaDuracion, int nuevaCantidadEstimada	public	Validar y modificar la información del artista. Si la duración no es mayor que 0, la cantidad estimada es negativa o algún texto está vacío, generará una <code>IllegalArgumentException</code> .
toString()	String	Ninguno	public	Devolver en forma de texto toda la información del artista para mostrarla al usuario.

Clase Festival

La clase Festival representa el festival universitario y administra sus escenarios y artistas. Su propósito es almacenar el arreglo de escenarios y el ArrayList de artistas, realizando las operaciones de registro, búsqueda, modificación, eliminación y cálculo.

Tabla de atributos

Propiedad	Tipo	Visibilidad	Propósito
nombre	String	private	Almacenar el nombre del festival universitario.
codigoidentificacion	String	private	Almacenar el código que identifica al festival.
nombreCoordinador	String	private	Almacenar el nombre de la persona encargada de coordinar el festival.
escenarios	Escenario[]	private	Almacenar los escenarios configurados mediante un arreglo básico de tamaño 5. Las posiciones no configuradas contendrán null.
artistas	ArrayList<Artista>	private	Almacenar una cantidad dinámica de artistas y permitir su registro, búsqueda, modificación y eliminación.

Clase Festival

Tabla de métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
Festival()	-	String nombre, String codigoidentificacion, String nombreCoordinador	public	Construir un festival con sus datos generales. Inicializará escenarios como un arreglo de tamaño 5 y artistas como un ArrayList vacío.
getNombre()	String	Ninguno	public	Obtener el nombre del festival.
getCodigoidentificacion()	String	Ninguno	public	Obtener el código de identificación del festival.
getNombreCoordinador()	String	Ninguno	public	Obtener el nombre del coordinador.
getCantidadPosicionesEscenarios()	int	Ninguno	public	Obtener el tamaño del arreglo de escenarios, que será 5. Permitirá recorrerlo sin exponer directamente el arreglo.
esPosicionValida()	boolean	int posicion	public	Determinar si una posición se encuentra entre 0 y escenarios.length - 1.
configurarEscenario()	boolean	int posicion, Escenario escenario	public	Almacenar un escenario en la posición indicada. Retornará false si la posición es inválida, se encuentra ocupada o el escenario recibido es null.
getEscenario()	Escenario	int posicion	public	Obtener el escenario de una posición. Retornará null si la

				posición es inválida o se encuentra vacía.
modificarEscenario()	boolean	int posicion, int nuevaCapacidad, String nuevoEstado	public	Modificar la capacidad y el estado del escenario de una posición. Retornará false si la posición es inválida o contiene null. Los valores incorrectos producirán una <code>IllegalArgumentExcep</code> tion.
retirarEscenario()	boolean	int posicion	public	Retirar el escenario de la posición indicada asignándole nuevamente null. Retornará false si la posición es inválida o ya está vacía.
contarEscenariosConfigurados()	int	Ninguno	public	Recorrer el arreglo y contar únicamente las posiciones que no contengan null.
contarEspaciosDisponibles()	int	Ninguno	public	Contar las posiciones del arreglo que todavía contengan null.
obtenerEscenarioMayorCapacidad()	Escenario	Ninguno	public	Recorrer el arreglo y devolver el escenario con mayor capacidad máxima. Retornará null si no existen escenarios configurados.
registrarArtista()	boolean	Artista artista	public	Agregar un artista al <code>ArrayList</code> si su código no está repetido. Retornará true si se registra y false si el código ya existe o el objeto es null.
buscarArtista()	Artista	String codigo	public	Recorrer el <code>ArrayList</code> y devolver el artista cuyo código coincida con el solicitado. Retornará null si no se encuentra.
modificarArtista()	boolean	String codigoActual, String nuevoCodigo, String nuevoNombreArtístico, String nuevoGeneroMusica	public	Buscar al artista mediante su código actual, comprobar que el nuevo código no esté repetido y actualizar su

		I, int nuevaDuracion, int nuevaCantidadEstimada		información. Retornará false si el artista no existe o el nuevo código pertenece a otro artista.
cancelarParticipacion()	boolean	String codigo	public	Buscar al artista por su código y eliminarlo del ArrayList. Retornará false si no se encuentra.
getCantidadArtistas()	int	Ninguno	public	Obtener la cantidad actual de artistas almacenados mediante <code>artistas.size()</code> .
getArtista()	Artista	int indice	public	Obtener un artista utilizando su índice dentro del ArrayList. Retornará null si el índice no es válido.
obtenerArtistaMayorDuracion()	Artista	Ninguno	public	Recorrer el ArrayList y devolver el artista con la presentación de mayor duración. Retornará null si no hay artistas.
obtenerArtistaMayorAsistencia()	Artista	Ninguno	public	Recorrer el ArrayList y devolver el artista con mayor cantidad estimada de asistentes. Retornará null si no hay artistas.
calcularPromedioDuracion()	double	Ninguno	public	Sumar las duraciones y calcular su promedio. Si el ArrayList está vacío, retornará 0.0 para evitar una división entre cero.

Clase Main

La clase Main funciona como el programa principal o driver program. Su propósito es mostrar el menú, solicitar información al usuario, llamar los métodos de las demás clases y manejar las excepciones que puedan ocurrir durante la ejecución.

Tabla de atributos

Propiedad	Tipo	Visibilidad	Propósito
scanner	Scanner	private static	Leer las entradas proporcionadas por el usuario durante la ejecución del programa.
festivalActual	Festival	private static	Mantener la instancia del festival que se administra actualmente. Será reemplazada al seleccionar la opción "Nuevo festival".

Clase Main

Tabla de métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
main()	void	String[] args	public static	Iniciar el programa, solicitar el festival inicial y ejecutar el menú. Utilizará un bloque finally para cerrar el Scanner al terminar.
crearFestival()	Festival	Ninguno	private static	Solicitar el nombre, código y coordinador, construir un nuevo festival y devolverlo. También se utilizará para reemplazar el festival actual.
ejecutarMenu()	void	Ninguno	private static	Mantener el programa funcionando mediante un ciclo y ejecutar las opciones del 1 al 13.

mostrarMenu()	void	Ninguno	private static	Mostrar las 13 opciones requeridas por el enunciado.
leerEntero()	int	String mensaje	private static	Solicitar un número entero. Manejará InputMismatchException, limpiará la entrada incorrecta y volverá a solicitar el dato.
leerTexto()	String	String mensaje	private static	Solicitar y devolver una entrada de texto proporcionada por el usuario.
configurarEscenario()	void	Ninguno	private static	Solicitar la posición y los datos de un escenario, construirlo y pedir a Festival que lo almacene. Manejará posibles IllegalArgumentException.
consultarEscenarios()	void	Ninguno	private static	Recorrer las cinco posiciones y mostrar solamente los escenarios configurados, junto con sus posiciones.
consultarEscenario()	void	Ninguno	private static	Solicitar una posición y mostrar el escenario almacenado. Informará si la posición es inválida o contiene null.
modificarEscenario()	void	Ninguno	private static	Solicitar una posición, nueva capacidad y nuevo estado para modificar un escenario existente.
retirarEscenario()	void	Ninguno	private static	Solicitar la posición de un escenario y retirarlo, dejando la posición con null.
registrarArtista()	void	Ninguno	private static	Solicitar la información de un artista, construirlo y registrarlo si su código no está repetido.
consultarArtistas()	void	Ninguno	private static	Recorrer y mostrar los artistas registrados. Informará si el ArrayList está vacío.
buscarArtista()	void	Ninguno	private static	Solicitar un código y mostrar el artista encontrado o informar que no existe.

modificarArtista()	void	Ninguno	private static	Solicitar el código actual y la nueva información del artista. Manejará valores inválidos y códigos repetidos.
cancelarParticipacion()	void	Ninguno	private static	Solicitar el código del artista y eliminarlo del ArrayList mediante Festival.
mostrarReporte()	void	Ninguno	private static	Mostrar todos los cálculos solicitados: escenarios configurados, espacios disponibles, escenario de mayor capacidad, cantidad de artistas, artistas destacados y promedio de duración.

Análisis

Preguntas y respuestas

7. Los objetos recibirán sus valores iniciales mediante sus constructores. El arreglo de escenarios iniciará con cinco posiciones null y el ArrayList de artistas estará vacío. Se validará que la capacidad y duración sean mayores que cero, que los asistentes no sean negativos, que los textos no estén vacíos y que no existan códigos de artistas repetidos.
8. Primero se verificará que la posición esté dentro de los límites del arreglo. Después se comparará su contenido con null: si contiene null, la posición está disponible; de lo contrario, contiene un escenario configurado.
9. La búsqueda se realizará recorriendo el ArrayList y comparando el código de cada artista. Para modificar, primero se buscará al artista y luego se actualizarán sus datos. Para eliminar, se localizará su posición y se utilizará el método remove(). También se verificará que el ArrayList no esté vacío.
10. Se manejará InputMismatchException cuando el usuario ingrese texto en lugar de un número, limpiando la entrada para continuar. Se utilizará IllegalArgumentException cuando la capacidad, duración o cantidad de asistentes sean inválidas. Estas excepciones se manejarán con try-catch en Main. El bloque finally cerrará el Scanner al terminar el programa. Las posiciones inválidas y los valores null se evitarán mediante validaciones previas o cuando algún dato de texto esté vacío.